

v3.0

Sugeneruota Doxygen 1.13.2

1 Klasės Indeksas	1
1.1 Klasės	1
2 Failo Indeksas	3
2.1 Failai	3
3 Klasės Dokumentacija	5
3.1 Vector< T > Klasė Šablonas	5
3.1.1 Smulkus aprašymas	7
4 Failo Dokumentacija	9
4.1 cFunctions.h	9
4.2 helper.h	9
4.3 helperFunctions.h	10
4.4 Vector.h	11
Rodyklė	15

skyrius 1

Klasės Indeksas

1.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Vector< T >	5
-----------------------------------	---

skyrius 2

Failo Indeksas

2.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

cFunctions.h	9
helper.h	9
helperFunctions.h	10
Vector.h	11

skyrius 3

Klasės Dokumentacija

3.1 Vector< T > Klasė Šablonas

```
#include <Vector.h>
```

Vieši Tipai

- using **value_type** = T
Member types.
- using **size_type** = size_t
- using **difference_type** = std::ptrdiff_t
- using **reference** = T&
- using **const_reference** = const T&
- using **pointer** = T*
- using **const_pointer** = const T*
- using **iterator** = T*
- using **const_iterator** = const T*

Vieši Metodai

- **Vector** ()
constructors
- **Vector** (size_t count)
- **Vector** (size_t count, const T &value)
- **Vector** (std::initializer_list< T > init)
- **Vector** (const **Vector** &other)
- **Vector** (**Vector** &&other) noexcept
- template<std::input_iterator InputIt>
Vector (InputIt first, InputIt last)
- **Vector** & **operator=** (const **Vector** &other)
- **Vector** & **operator=** (**Vector** &&other) noexcept
- **Vector** & **operator=** (std::initializer_list< T > init)
- void **assign** (size_t count, const T &value)
member functions
- template<std::input_iterator InputIt>
void **assign** (InputIt first, InputIt last)

- void **assign** (std::initializer_list< T > ilist)
- bool **empty** () const noexcept

capacity

- size_t **size** () const noexcept
- size_t **max_size** () const noexcept
- size_t **capacity** () const noexcept
- void **reserve** (size_t new_cap)
- void **shrink_to_fit** ()
- void **clear** ()

modifiers

- iterator **insert** (iterator pos, const T &value)
- iterator **insert** (iterator pos, size_t count, const T &value)
- template<std::input_iterator InputIt>
iterator **insert** (iterator pos, InputIt first, InputIt last)
- iterator **insert** (iterator pos, T &&value)
- iterator **insert** (iterator pos, std::initializer_list< T > ilist)
- iterator **erase** (iterator pos)
- iterator **erase** (const_iterator pos)
- iterator **erase** (iterator first, iterator last)
- iterator **erase** (const_iterator first, const_iterator last)
- void **push_back** (const T &value)
- void **push_back** (T &&value)
- void **pop_back** ()
- template<typename... Args>
iterator **emplace** (iterator pos, Args &&... args)
- template<typename... Args>
void **emplace_back** (Args &&... args)
- void **resize** (size_t new_size)
- void **resize** (size_t new_size, const T &value)
- void **swap** (Vector &other) noexcept
- T & **at** (size_t index)

element access

- const T & **at** (size_t index) const
- T & **front** ()
- T & **back** ()
- const T & **front** () const
- const T & **back** () const
- T & **operator[]** (size_t index)
- const T & **operator[]** (size_t index) const
- T * **data** () noexcept
- const T * **data** () const noexcept
- iterator **begin** () noexcept

iterators

- const_iterator **begin** () const noexcept
- const_iterator **cbegin** () const noexcept
- iterator **end** () noexcept
- const_iterator **end** () const noexcept
- const_iterator **cend** () const noexcept
- std::reverse_iterator< iterator > **rbegin** () noexcept
- std::reverse_iterator< const_iterator > **rbegin** () const noexcept
- std::reverse_iterator< const_iterator > **crbegin** () const noexcept
- std::reverse_iterator< iterator > **rend** () noexcept
- std::reverse_iterator< const_iterator > **rend** () const noexcept
- std::reverse_iterator< const_iterator > **crend** () const noexcept

3.1.1 Smulkus aprašymas

```
template<typename T>  
class Vector< T >
```

this is my [Vector](#) class

Dokumentacija šiai klasei sugeneruota iš šio failo:

- Vector.h

skyrius 4

Failo Dokumentacija

4.1 cFunctions.h

```
00001 #ifndef CFUNCTIONS_H
00002 #define CFUNCTIONS_H
00003
00004 #include "helperFunctions.h"
00005
00006 Stud CreateStudent(int stCon);
00007 void FullDataProccess(string filename1, string filename2, string filename3, string filename4, string
    filename5);
00008 void FullContainerTest(string filename1, string filename2, string filename3, string filename4, string
    filename5);
00009
00010
00011 #include "cFunctions.tpp"
00012
00013 #endif
```

4.2 helper.h

```
00001 #ifndef HELPER_H
00002 #define HELPER_H
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <string>
00007 // #include <vector>
00008 #include <sstream>
00009 #include <optional>
00010 #include <limits>
00011 #include <algorithm>
00012 #include <fstream>
00013 #include <chrono>
00014 #include <exception>
00015 #include <random>
00016 #include <sstream>
00017 #include <list>
00018 #include <deque>
00019 #include "Vector.h"
00020
00021 using std::cout;
00022 using std::cin;
00023 using std::endl;
00024 using std::string;
00025 using std::ifstream;
00026 using std::ofstream;
00027 using std::cerr;
00028 using std::getline;
00029 using namespace std::chrono;
00030 using std::sort;
00031 using std::optional;
00032 using std::stringstream;
00033 using std::list;
00034 using std::deque;
00035
```

```

00036
00037 template <typename T>
00038 optional<T> string_to(const string & s);
00039
00040 int CheckInt(const string& text, int max = 0);
00041 int TypeInt(const string& text, int max = 0);
00042 string TypeString(const string& prompt);
00043 double Median(Vector<int> numbers);
00044 double Average(Vector<int> numbers);
00045
00046 extern int MAX_GRADES;
00047
00048 class Zmogus {
00049     protected:
00050         string pavarde;
00051         string vardas;
00052
00053     public:
00054         Zmogus() {pavarde = "Pavardenis"; vardas = "Vardenis";};
00055         Zmogus(const Zmogus& other) : pavarde(other.pavarde), vardas(other.vardas) {}
00056         Zmogus(Zmogus&& other) : pavarde(std::move(other.pavarde)), vardas(std::move(other.vardas)) {}
00057         virtual void KiekGavauIsEgz() = 0;
00058 };
00059
00060 class Stud : public Zmogus {
00061     private:
00062         int egz;
00063         Vector<int> ndVector;
00064         int* nd;
00065         int nd_count;
00066         float galutinisVid;
00067         float galutinisMed;
00068
00069     public:
00070
00071         Stud();
00072         virtual ~Stud();
00073
00074         //getters
00075         inline string getPavarde() const { return pavarde; }
00076         inline string getVardas() const { return vardas; }
00077         inline int getEgz() const { return egz; }
00078         inline Vector<int> getNdVector() const { return ndVector; }
00079         inline int* getNd() const { return nd; }
00080         inline int getNdCount() const { return nd_count; }
00081         inline float getGalutinisVid() const { return galutinisVid; }
00082         inline float getGalutinisMed() const { return galutinisMed; }
00083
00084         //setters
00085         inline void setPavarde(const string& pavarde) { this->pavarde = pavarde; }
00086         inline void setVardas(const string& vardas) { this->vardas = vardas; }
00087         inline void setEgz(int egz) { this->egz = egz; }
00088         inline void setNdVector(const Vector<int>& ndVector) { this->ndVector = ndVector; }
00089         inline void setNd(int* nd, int count) {
00090             delete[] this->nd;
00091             this->nd = new int[count];
00092             std::copy(nd, nd + count, this->nd);
00093             this->nd_count = count;
00094         }
00095         inline void setGalutinisVid(float galutinisVid) { this->galutinisVid = galutinisVid; }
00096         inline void setGalutinisMed(float galutinisMed) { this->galutinisMed = galutinisMed; }
00097
00098         void clearNdVector() {this->ndVector.clear(); }
00099         void addNd(int nd) {this->ndVector.push_back(nd); }
00100
00101         //rule of 5
00102         Stud(const Stud& other);
00103         Stud& operator=(const Stud& other);
00104         Stud(Stud&& other) noexcept;
00105         Stud& operator=(Stud&& other) noexcept;
00106
00107         friend std::ostream& operator<<(std::ostream& os, const Stud& stud);
00108         friend std::istream& operator>>(std::istream& is, Stud& student);
00109
00110         void KiekGavauIsEgz() override {cout << "Is egzamino gavau" << egz << endl;}
00111 };
00112
00113 void MethodTest();
00114
00115 #endif

```

4.3 helperFunctions.h

```
00001 #ifndef HELPFUNCTIONS_H
```

```

00002 #define HELPFUNCTIONS_H
00003
00004 #include "helper.h"
00005
00006 int RandInt(int min, int max);
00007 int RandIntWSeed(int min, int max, std::mt19937 generator);
00008 string GenName();
00009 string GenSurname();
00010 void GenFile(string filename, int amount);
00011 void GenFiles();
00012 int CountWordsInLine(const string& line);
00013 void Pause();
00014
00015 #endif // HELPFUNCTIONS_H

```

4.4 Vector.h

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <cstddef>
00005 #include <initializer_list>
00006 #include <stdexcept>
00007 #include <limits>
00008 #include <iterator>
00009 #include <iostream>
00010 #include <utility>
00011 #include <concepts>
00012 #include <iterator>
00013
00017
00018 template <typename T>
00019 class Vector {
00020 private:
00021     T* dataVar;
00022     size_t capacityVar;
00023     size_t sizeVar;
00024
00025 public:
00026
00028     using value_type          = T;
00029     using size_type           = size_t;
00030     using difference_type     = std::ptrdiff_t;
00031     using reference           = T&;
00032     using const_reference     = const T&;
00033     using pointer             = T*;
00034     using const_pointer       = const T*;
00035     using iterator            = T*;
00036     using const_iterator      = const T*;
00037
00038
00040     // Constructors
00041     Vector(); // Default constructor
00042     explicit Vector(size_t count); // Fill constructor
00043     Vector(size_t count, const T& value); // Fill with value
00044     Vector(std::initializer_list<T> init); // Initializer list constructor
00045     Vector(const Vector& other); // Copy constructor
00046     Vector(Vector&& other) noexcept; // Move constructor
00047     template <std::input_iterator InputIt>
00048     Vector(InputIt first, InputIt last);
00049
00050     // Destructor
00051     ~Vector();
00052
00053     // Assignment operators
00054     Vector& operator=(const Vector& other); // Copy assignment
00055     Vector& operator=(Vector&& other) noexcept; // Move assignment
00056     Vector& operator=(std::initializer_list<T> init); // Initializer list assignment
00057
00059     void assign(size_t count, const T& value);
00060     template <std::input_iterator InputIt>
00061     void assign(InputIt first, InputIt last);
00062     void assign(std::initializer_list<T> ilist);
00064     bool empty() const noexcept;
00065
00066     size_t size() const noexcept;
00067
00068     size_t max_size() const noexcept;
00069
00070     size_t capacity() const noexcept;
00071
00072     void reserve(size_t new_cap);
00073

```

```

00074     void shrink_to_fit();
00075
00077     void clear();
00078
00079     iterator insert(iterator pos, const T& value);
00080     iterator insert(iterator pos, size_t count, const T& value);
00081     template <std::input_iterator InputIt>
00082     iterator insert(iterator pos, InputIt first, InputIt last);
00083     iterator insert(iterator pos, T&& value);
00084     iterator insert(iterator pos, std::initializer_list<T> ilist);
00085
00086     iterator erase(iterator pos);
00087     iterator erase(const_iterator pos);
00088     iterator erase(iterator first, iterator last);
00089     iterator erase(const_iterator first, const_iterator last);
00090
00091     void push_back(const T& value);
00092     void push_back(T&& value);
00093
00094     void pop_back();
00095
00096     template <typename... Args>
00097     iterator emplace(iterator pos, Args&&... args);
00098
00099     template <typename... Args>
00100     void emplace_back(Args&&... args);
00101
00102     void resize(size_t new_size);
00103     void resize(size_t new_size, const T& value);
00104
00105     void swap(Vector& other) noexcept;
00106
00108     T& at(size_t index);
00109     const T& at(size_t index) const;
00110
00111     T& front();
00112     T& back();
00113     const T& front() const;
00114     const T& back() const;
00115
00116     T& operator[](size_t index);
00117     const T& operator[](size_t index) const;
00118
00119     T* data() noexcept;
00120     const T* data() const noexcept;
00121
00122
00124     iterator begin() noexcept;
00125     const_iterator begin() const noexcept;
00126     const_iterator cbegin() const noexcept;
00127
00128     iterator end() noexcept;
00129     const_iterator end() const noexcept;
00130     const_iterator cend() const noexcept;
00131
00132     std::reverse_iterator<iterator> rbegin() noexcept;
00133     std::reverse_iterator<const_iterator> rbegin() const noexcept;
00134     std::reverse_iterator<const_iterator> crbegin() const noexcept;
00135
00136     std::reverse_iterator<iterator> rend() noexcept;
00137     std::reverse_iterator<const_iterator> rend() const noexcept;
00138     std::reverse_iterator<const_iterator> crend() const noexcept;
00139 };
00140
00142 // Non-member function declarations:
00143 template <typename T>
00144 bool operator==(const Vector<T>& lhs, const Vector<T>& rhs);
00145
00146 template <typename T>
00147 bool operator!=(const Vector<T>& lhs, const Vector<T>& rhs);
00148
00149 template <typename T>
00150 bool operator<(const Vector<T>& lhs, const Vector<T>& rhs);
00151
00152 template <typename T>
00153 bool operator<=(const Vector<T>& lhs, const Vector<T>& rhs);
00154
00155 template <typename T>
00156 bool operator>(const Vector<T>& lhs, const Vector<T>& rhs);
00157
00158 template <typename T>
00159 bool operator>=(const Vector<T>& lhs, const Vector<T>& rhs);
00160
00161 template <typename T>
00162 void swap(Vector<T>& a, Vector<T>& b) noexcept;
00163
00164 #include "Vector.tpp"

```



```
00165  
00166 #endif
```


Rodyklė

Vector< T >, 5